

Entrega 2 – Álgebra Linear, Vetores e Geometria Analítica

Aplicação de Transformações Lineares em Imagens Digitais
Projeto Interdisciplinar – Lideranças Empáticas (LE) – Scanner AI | 5CCOMP – 2026

Curso	Ciência da Computação
Turma	5CCOMP – 5º Semestre 2026
Projeto	Lideranças Empáticas (LE) – Scanner AI
Disciplina	Álgebra Linear, Vetores e Geometria Analítica
Entregável	Entrega 2 – Transformações Lineares em Imagens Digitais
Integrantes	Antonio Petri Leonardo Santos Lucas Gutierrez Vitor Kanashiro

Na Entrega 1, a imagem *Arroz.jpg* foi carregada com OpenCV e convertida para o espaço RGB. Cada pixel (x, y) foi representado como vetor [R, G, B] e exportado para *matriz_imagem_alimento.csv*. A reconstrução comprovou que a informação visual está integralmente codificada na estrutura matricial. Nesta entrega, aplicamos operações de Álgebra Linear sobre as coordenadas espaciais, alterando a disposição dos pixels sem modificar os valores de cor.

Coordenadas Homogêneas

Para representar transformações lineares e translações de forma unificada, cada ponto (x, y) é representado como vetor [x, y, 1]^T em R³ (coordenadas homogêneas). Assim, qualquer combinação de rotação, escalonamento, cisalhamento e translação é expressa por uma única multiplicação de matriz 3x3:

[x']

[y']

[1]

=

[M11 M12 M13]

[M21 M22 M23]

[0 0 1]

×

[x]

[y]

[1]

Matrizes de Transformação

A tabela abaixo apresenta as cinco matrizes com seus parâmetros e determinantes:

Transformação	Matriz (3×3)	Parâmetros	det(M)
Escalonamento	[0.5 0 0] [0 0.5 0] [0 0 1]	Sx = 0.5 Sy = 0.5	0.25
Rotação (45°)	[cos θ -sin θ tx] [sin θ cos θ ty] [0 0 1]	θ = 45° (centralizada)	1.00
Cisalhamento	[1 0.5 0] [0 1 0] [0 0 1]	shx = 0.5	1.00
Reflexão Horiz.	[-1 0 w-1] [0 1 0] [0 0 1]	Eixo vertical central	-1.00
Projeção (Colapso Dim.)	[1 0 0] [0 0 0]	Projeta y -> 0	0.00

Transformação	Matriz (3×3)	Parâmetros	det(M)
	$\begin{bmatrix} 0 & 0 & 1 \\ \end{bmatrix}$		

Implementação em Python

Definição das matrizes

```
import numpy as np, cv2, matplotlib.pyplot as plt
img = cv2.cvtColor(cv2.imread('Arroz.jpg'), cv2.COLOR_BGR2RGB)
h, w, _ = img.shape

# 1. Escalonamento (50%)
M_escala = np.array([[0.5, 0, 0], [0, 0.5, 0], [0, 0, 1]])

# 2. Rotacao 45 graus (centralizada no canvas)
theta = np.radians(45); c, s = np.cos(theta), np.sin(theta)
diag = int(np.ceil(np.sqrt(w**2 + h**2))); cx, cy = w/2, h/2
M_rot = np.array([[c, -s, -cx*c+cy*s+diag/2], [s, c, -cx*s-cy*c+diag/2], [0, 0, 1]])

# 3. Cisalhamento (shx = 0.5)
M_cis = np.array([[1, 0.5, 0], [0, 1, 0], [0, 0, 1]])

# 4. Reflexao horizontal
M_ref = np.array([[-1, 0, w-1], [0, 1, 0], [0, 0, 1]])

# 5. Projecao - colapso dimensional (det = 0)
M_proj = np.array([[1, 0, 0], [0, 0, 0], [0, 0, 1]])
```

Mapeamento Inverso (Backward Mapping)

Para evitar buracos na imagem transformada, usa-se o **mapeamento inverso**: para cada pixel (x', y') da imagem destino, calcula-se a posição de origem aplicando M^{-1} . Quando $\det(M) = 0$, a matriz é singular — aplica-se o mapeamento direto, resultando no colapso dimensional.

```
def aplicar_transformacao(img, M, novo_w=None, novo_h=None):
    src_h, src_w = img.shape[:2]
    dst_h = novo_h or src_h; dst_w = novo_w or src_w
    img_out = np.zeros((dst_h, dst_w, 3), dtype=np.uint8)
    if abs(np.linalg.det(M)) < 1e-10: # Matriz singular
        yy, xx = np.meshgrid(range(src_h), range(src_w), indexing='ij')
        coords = np.stack([xx.ravel(), yy.ravel(), np.ones(src_h*src_w)])
        dst = M @ coords
        xd = np.round(dst[0]).astype(int); yd = np.round(dst[1]).astype(int)
        mask = (xd>=0) & (xd<dst_w) & (yd>=0) & (yd<dst_h)
        img_out[yd[mask], xd[mask]] = img[yy.ravel()[mask], xx.ravel()[mask]]
    else: # Mapeamento inverso
        M_inv = np.linalg.inv(M)
        yy, xx = np.meshgrid(range(dst_h), range(dst_w), indexing='ij')
        coords = np.stack([xx.ravel(), yy.ravel(), np.ones(dst_h*dst_w)])
        src = M_inv @ coords
        xs = np.round(src[0]).astype(int); ys = np.round(src[1]).astype(int)
        mask = (xs>=0) & (xs<src_w) & (ys>=0) & (ys<src_h)
        img_out[yy.ravel()[mask], xx.ravel()[mask]] = img[ys[mask], xs[mask]]
    return img_out
```

Análise Conceitual das Transformações

Transformação	det(M)	Efeito Geométrico	Propriedades Algébricas
Escalonamento (Sx=Sy=0.5)	0.25	Reduz a imagem a 50% em cada eixo. Área resultante = 25% da original.	Autovalores: Sx e Sy. Preserva paralelismo e proporções internas.
Rotação (θ = 45°)	1.00	Gira a imagem 45° anti-horário. Não altera forma nem tamanho.	M ortogonal: M ⁻¹ = M ^T . Autovalores complexos: e ^{iθ} e e ^{-iθ} .
Cisalhamento (shx = 0.5)	1.00	Converte retângulo em paralelogramo. Preserva área mas altera ângulos.	Autovalor duplo = 1. Apenas 1 autovetor real no eixo x.
Reflexão Horizontal	-1.00	Espelha em torno do eixo vertical. Inverte orientação (handedness).	Isometria: preserva distâncias. Autovalores: +1 e -1.
Projeção (Colapso Dim.)	0.00	Projeta todos os pixels em y=0. Perda irreversível de informação.	Matriz singular: sem inversa. dim(ker)=1: toda direção y vai a zero.

Colapso Dimensional

A transformação de projeção ilustra um conceito central da Álgebra Linear: quando **det(M) = 0**, a matriz é **singular** e não possui inversa. O núcleo (kernel) tem dimensão não nula — toda a direção y é mapeada para zero. Informações distintas tornam-se indistinguíveis, tornando a recuperação da imagem original matematicamente impossível.

Na prática, esse fenômeno ocorre em Visão Computacional quando câmeras capturam cenas com perspectiva extrema ou oclusão total — situações onde parte da informação visual é irrecuperável.

Conclusão

Este trabalho demonstrou que **transformações lineares** são mecanismos fundamentais da computação gráfica e do processamento de imagens com IA. O determinante da matriz revela propriedades essenciais: com $\det \neq 0$ a transformação é invertível e preserva (ou escala) o espaço; com $\det = 0$ ocorre colapso dimensional e perda irreversível de informação. No projeto **Lideranças Empáticas – Scanner AI**, esses fundamentos embasam o pré-processamento de imagens do pipeline de Visão Computacional para classificação dos alimentos arrecadados.